Skyline Asset Management

- **Customer** – Matan Dan On
- **Designer** – Lior Sheiner
- **Reviewer** – Noa Akerman

Main Use Case

Skyline Asset Management handles in-house leasing, maintenance, and accounting for a real estate portfolio. A property manager enters lease details, tenant information, and tracks incoming rent payments. The system also tracks maintenance work orders, contractor details, and repair costs. Management uses the system to monitor overall cash flow, consolidate liabilities, and generate financial reports.

Customer Requirements

1. The system should store property details, unit numbers, and specific unit amenities for marketing purposes.
2. The system should store tenant contact information and manage active leases, including those with multiple roommates.
3. The system should create maintenance work orders containing repair status, outside contractor details, and billed amounts.
4. The system should unify all outgoing business debts (supplier payments and employee salaries) into a single liabilities list.
5. The system should track incoming rent payments and hold a consolidated security deposit for each lease.
6. The system should track the building's lifecycle, including purchase details and age, for depreciation purposes.

Designer Understanding of the Use Case

As the designer, I understand that the main challenges are centralizing financial records and standardizing property data. I understand that liabilities must be unified into a single table for management. In addition, the system needs to handle multi-tenant leases and link maintenance tickets to external contractors without duplicating contact information or violating database normalization.

Discussion 1 – Property Information and Entities

Discussion between Customer and Designer

- **Customer:** I need the system to output the full property address and its amenities in one big text block to easily copy and paste this into listings.

- **Designer:** I explained that atomic columns for addresses and a dedicated Amenities linking table improve query efficiency, and a view for concatenating the full address and amenities for a given unit.

  Discussion between Designer and Reviewer
- **Designer:** I suggested using a separate Entities table to link to Properties and a view for easy copy and pasteable block of text for address and amenities.
- **Reviewer:** A view for concatenating the full address and amenities serves as an easy and normalization friendly solution for the need, note that a linking unit amenities table also has to be created.

Final Decision We agreed to use atomic address columns and a Unit_Amenities linking table. To fulfill the customers' need for easy copy and paste we will create a Property_listings view to generate the block of text on demand.

Discussion 2 – Financials and Building Tracking

Discussion between Customer and Designer

- **Customer:** We wanted all outgoing money in one unified liabilities list, split security deposits (damages vs. utilities), and a static building age column (for tax purposes).
- **Designer:** I will create the unified Liabilities table, I will also include split deposits on the lease, but a static 'age' column seems like a bad idea.

Discussion between Designer and Reviewer

- **Designer:** The customers' request would mean storing the building's current age as an integer, and separating security deposit types.
- **Reviewer:** I advised against a static age column as it'll require annual updates, instead I suggest having a static build year column which can be used for age calculations. I also advised consolidating the security deposit into a single total amount to handle future policy changes flexibly.

Final Decision

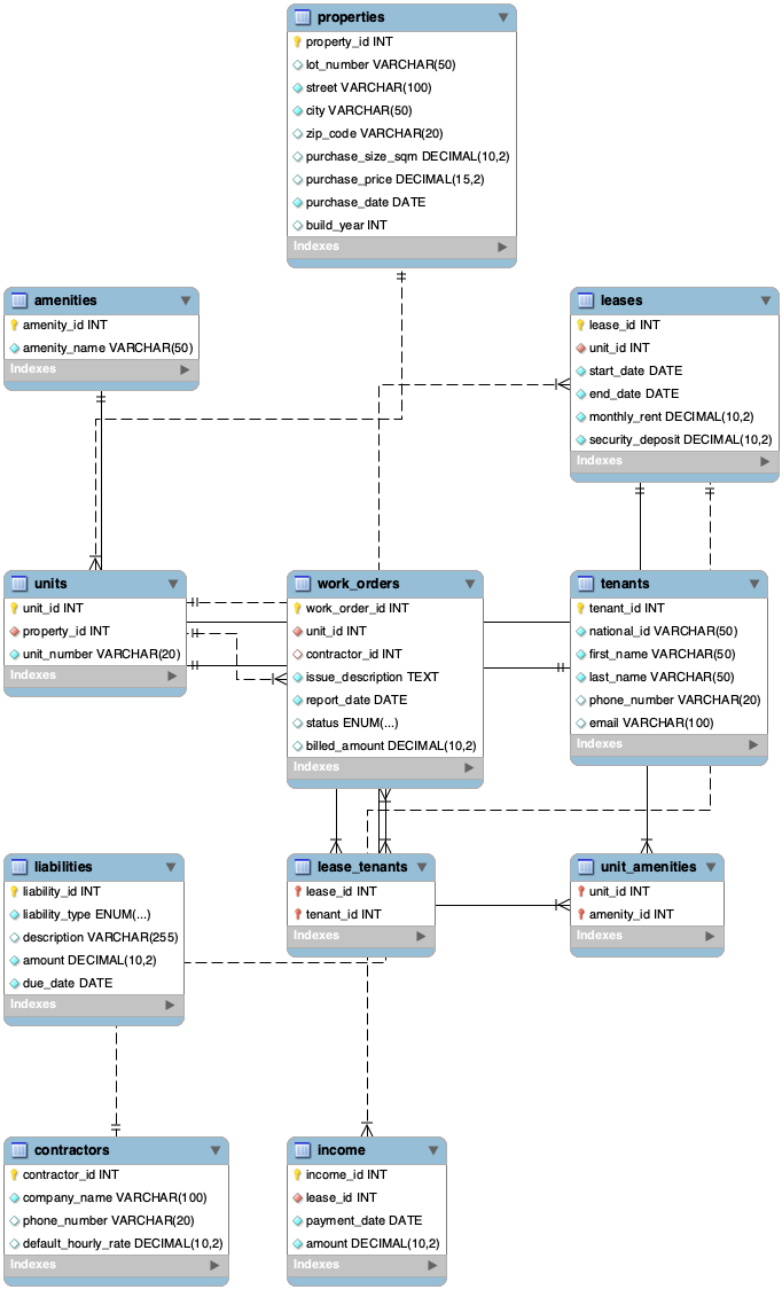
We agreed to implement the unified Liabilities table. We replaced the static age column with a build_year column for dynamic calculation, and we consolidated the security deposit into a single column in the Leases table.

Final Agreed System Design

The final database design includes separate tables for properties, units, amenities, tenants, leases, income, liabilities, contractors, and work orders. The tables are connected using primary keys and foreign keys in order to maintain organized and reliable data. The design supports:

- Dynamic age calculation and centralized contractor management.

- Multi-tenant leases and strictly formatted contact information.
- Unified liability accounting and consolidated deposits.



Reviewer Summary

As the reviewer, I verified that the final database design fulfills customer requirements while strictly adhering to normalization principles. Key improvements include enforcing atomic data structures (supported by a View), consolidating financial fields for flexibility, applying robust check constraints, and dynamically calculating building metrics. Overall, the design provides an organized, scalable, and reliable solution for Skyline Asset Management.